

K1. Тріаж невідомої плати

Плата в руках, походження невідоме. Порядок дій — згори вниз. Не перестрибуйте: кожен крок відсікає варіанти для наступного.

1. Дивитися, не вмикати

Прочитати напис на металевій кришці модуля — це головне джерело істини:

Напис на модулі	Чип	Що це значить
ESP32-WROOM-32	ESP32 classic	двоядерний Xtensa, без PSRAM
ESP32-WROVER	ESP32 classic	1. PSRAM; вона займає частину пінів
ESP32-S3-WROOM-1	ESP32-S3	двоядерний, native USB
ESP32-C3-MINI-1	ESP32-C3	одноядерний RISC-V
ESP8266 / ESP-12	не ESP32	інша архітектура, інший тулчейн

Далі — очима, без живлення: чи цілий USB-роз’єм, чи не здутий стабілізатор, чи немає слідів припою між сусідніми пінами, чи не обвуглене щось.

2. Живлення — до першого підключення

НЕЗВОРОТНЕ

Логіка ESP32 — 3.3 В. П’ять вольтів на будь-який GPIO вбивають пін або чип. На платі є пін 5V/VIN — він іде на стабілізатор, а не в чип; це єдиний вхід, куди 5 В подавати можна.

Мультиметром у режимі прозвонки, живлення вимкнене: короткого замикання не має бути ні між 3V3 і GND, ні між 5V і GND. Дзвенить — далі не вмикати, шукати замикання.

3. Підключити і подивитися лог

USB-кабель має бути **data**, а не тільки для заряджання. Термінал на 115200 бод, потім коротко натиснути EN (RST).

- З’явився осмислений текст → чип живий, є прошивка. Далі — картка K6.
- Текст «кракозябрами» → не та швидкість. Читається на **74880** — це ESP8266, не ESP32 (у нього ROM говорить саме так).
- Один рядок, тиша, знову той самий рядок → boot loop. Далі — картка K7.
- Порожньо → чип не стартує або немає порту. Далі — картка K3.

4. Опитати чип

```
esptool --port /dev/ttyUSB0 chip-id
esptool --port /dev/ttyUSB0 flash-id
```

chip-id називає сімейство і ревізію — звірити з написом на модулі. flash-id називає обсяг флешу: менший за очікуваний означає клон із перемаркованим модулем.

УВАГА

Якщо esptool не бачить чип — це ще не вирок платі. Спершу картка K4: вхід у download mode вручну кнопками BOOT + EN.

5. Зберегти стан — до будь-яких змін

Перш ніж прошивати, стирати чи міняти конфігурацію — зняти повний дамп флешу. Далі — картка K2. Це єдина операція, після якої плату завжди можна повернути в початковий стан.

Дерево рішень

Що маємо	Що з цим робити
Чип відповідає, лог осмислений	робочий пристрій → K2, потім K6
Чип відповідає, лог порожній	немає прошивки → K5
Чип відповідає, boot loop	K7, потім K8
Чип не відповідає, порт є	K4 (download mode вручну)
Чип не відповідає, порту немає	K3 (кабель, драйвер, права)
K3 по живленню	ремонт, не прошивка → K13

K2. Збереження стану перед втручанням

Усе, що нижче, робиться **до** першої зміни. Після erase-flash або перепрошивки повернути початковий стан можна лише з того, що ви встигли зняти. Двадцять хвилин зараз — або незворотна втрата.

1. Фото

Обидві сторони плати, з такої відстані, щоб читалися написи на чипах. Окремо — кожен роз'єм із під'єднаними дротами, **до** від'єднання. Окремо — положення всіх перемичок і мікроперемичаків.

Знімати при доброму світлі й без спалаху: спалах засвічує лазерне маркування на чипах, а саме воно потім знадобиться.

2. Схема з'єднань

Записати, який дріт куди йде. Не «синій до плати», а: синій → GPIO4, червоний → 3V3, чорний → GND. Кольори дротів у чужих виробках не означають нічого — покладатися можна лише на записане.

3. Вимірювання

До подачі живлення, мультиметром у режимі прозвонки: опір між 3V3 і GND, між 5V і GND. Записати обидва значення.

Під живленням: реальна напруга на 3V3 і на вході. Записати. Ці чотири числа — єдина база для порівняння, якщо потім щось зміниться.

4. Дамп флешу

Головне. Обсяг флешу спершу дізнатися, інакше дамп буде неповний:

```
esptool --port /dev/ttyUSB0 flash-id
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump-2026-08-26.bin
```

ALL читає рівно стільки, скільки є на чипі. Якщо esptool цього не підтримує — підставити обсяг явно: 0x400000 для 4 МБ, 0x800000 для 8 МБ.

Перевірити результат одразу: розмір файлу має точно дорівнювати обсягу флешу. Файл, менший за очікуваний, — обірваний дамп, а не «стисненіший».

5. Куди це кладеться

Один каталог на пристрій, назва — дата й що це. Всередині: фото, текстовий файл із записами, дамп. Дамп без записів через місяць нічого не означає.

НЕЗВОРОТНЕ

Дамп, що лежить лише на робочому ноутбуку, — це не резервна копія. Скопіювати в друге місце **до** того, як почнете втручатися.

К3. Підключення до ПК: кабель → драйвер → порт → права

Порт не з'явився. Причина майже завжди в одному з чотирьох місць, і шукати їх треба саме в цьому порядку — від найчастішого до найрідшого.

1. Кабель

Найчастіша причина. Дешеві кабелі з комплектів до павербанків мають лише дві жили живлення і **жодних даних**. Зовні не відрізняються.

Перевірка: під'єднати цим кабелем будь-який пристрій, що точно визначається (телефон, флешку). Не визначився — кабель для заряджання, взяти інший.

2. Живлення і сам чип

Індикатор живлення на платі горить? Ні — кабель, роз'єм або плата. Роз'єм micro-USB на дешевих платах відривається разом із площадками; поворушити його при увімкненому живленні: блимає — тріщина в пайці.

3. Драйвер USB-UART мосту

Що саме стоїть на платі — видно за маркуванням меншого чипа біля роз'єма:

Маркування	Міст	Драйвер
CP2102, CP2102N	Silicon Labs	Windows: з сайту SiLabs. Linux: у ядрі
CH340, CH341	WCH	Windows: з сайту WCH. Linux: у ядрі
CH9102, CH9102F	WCH	Windows: окремий, не той, що для CH340
немає окремого чипа	native USB S3 C3	драйвер не потрібен

Windows: Диспетчер пристроїв → жовтий знак оклику означає драйвер, а не залізо. Linux: `dmesg | tail` одразу після під'єднання показує, чи ядро взагалі побачило пристрій.

4. Права на порт (Linux)

Порт `/dev/ttyUSB0` є, але програма пише «Permission denied» — це права, а не несправність:

```
sudo usermod -aG dialout $USER
```

Далі — **перезайти в систему**. Без цього група не застосується, і здаватиметься, що команда не спрацювала.

УВАГА

Не запускати прошивку через `sudo`, щоб «обійти права». Це працює один раз і залишає по собі файли проекту, що належать `root`, — далі збирання ламається у неочевидний спосіб.

Якщо порт є, а зв'язку немає

Порт зайнятий іншою програмою: відкритий монітор, Arduino IDE, `screen`. Одночасно порт тримає лише один процес. Закрити все зайве і повторити.

K4. Download mode вручну (BOOT + EN)

Плата не переходить у режим прошивки сама. Це нормальна ситуація для частини плат, а не ознака несправності.

Що таке download mode

Щоб прийняти прошивку, чип має стартувати не в застосунок, а в ROM-бутлоадер. Вибір робиться **в момент скидання** за станом strapping-піна.

classic **S3** Вирішує GPIO:

- GPIO вільний (підтягнутий вгору) → звичайний старт застосунку;
- GPIO притиснутий до землі → download mode.

C3 Вирішує пара пінів: до землі притискається GPIO9, а GPIO8 при цьому має лишатися високим. Кнопка BOOT на платах C3 діє саме на GPIO9, тому послідовність нижче не змінюється.

Кнопка BOOT (іноді I00, FLASH) саме притискає GPIO до землі. Кнопка EN (іноді RST, RESET) — скидання.

Послідовність

Порядок обов'язковий: стан GPIO читається один раз, у момент відпускання скидання.

1. Натиснути і **тримати** BOOT.
2. Не відпускаючи BOOT, коротко натиснути й відпустити EN.
3. Зачекати півсекунди.
4. Відпустити BOOT.

Ознака успіху — у моніторі порту рядок виду waiting for download. Якщо монітор закритий, просто запустити прошивку: вона має початися без «Failed to connect».

Чому авторесет не спрацював

На платі є схема, що смикає GPIO і EN сигналами DTR/RTS USB-мосту. Вона не універсальна і не працює, коли:

- на GPIO або GPIO2 навішано зовнішню обв'язку, яка їх утримує;
- плата без такої схеми взагалі (голі модулі, частина клонів);
- живлення просідає під час скидання — конденсатори не встигають;
- драйвер мосту не керує DTR/RTS (трапляється на CH9102 у Windows).

Плати без кнопок

Голий модуль або плата без кнопок: перемичкою або пінцетом замкнути GPIO на GND, потім коротко замкнути EN на GND і відпустити, далі прибрати перемичку з GPIO. Послідовність та сама.

УВАГА

classic На платах ESP32-CAM кнопки BOOT немає взагалі: GPIO з'єднується з GND перемичкою на самій платі. Після прошивки перемичку обов'язково зняти, інакше плата так і лишиться в download mode.

Якщо не допомогло

Понизити швидкість: -baud 115200. Спробувати інший USB-порт напряду, без хаба. Зняти всі дроти зі strapping-пінів — під час старту вони мають бути вільні. **classic** Це GPIO, GPIO2, GPIO5, GPIO12, GPIO15; **S3** GPIO, GPIO3, GPIO45, GPIO46; **C3** GPIO2, GPIO8, GPIO9 (картка K9).

К5. Прошивка готового образу

Прошити зібраний кимось образ, не збираючи проект.

Три образи і їхні адреси

Повна прошивка ESP-IDF — це три файли, кожен на своїй адресі:

Файл	Що це	classic, S2	S3, C3, C6, H2	P4, C5, H4
bootloader.bin	другий бутлоадер	0x1000	0x0	0x2000
partition-table.bin	таблиця розділів	0x8000	0x8000	0x8000
застосунок .bin	сама програма	0x10000	0x10000	0x10000

НЕЗВОРОТНЕ

Різниця в адресі бутлоадера — найчастіша причина «прошилося без помилок, але плата мовчить». Команда з інструкції для ESP32 classic кладе бутлоадер S3 на 0x1000, тобто не туди. Спершу визначити чип (картка K1), потім брати адресу.

Команда

```
esptool --port /dev/ttyUSB0 --baud 460800 write-flash -z \  
0x1000 bootloader.bin \  
0x8000 partition-table.bin \  
0x10000 app.bin
```

⚠ У команді вище стоїть 0x1000 — адреса **classic i S2**. Для решти чипів перший рядок інший: див. таблицю вище. Правила «що новіше, то ближче до нуля» немає — адресу задає ROM чипа.

-z вмикає стиснення — швидше і безпечніше для довгих кабелів. Не з'єднається — знизити до --baud 115200.

Якщо образ **один** файл (зібраний через merge-bin), адреса завжди 0x0, незалежно від сімейства чипа: зсуви вже всередині файлу.

```
esptool --port /dev/ttyUSB0 write-flash 0x0 merged.bin
```

Синтаксис: v5 проти v4

Команди вище — для esptool v5. Якщо у вас v4 (іде з ESP-IDF 5.x), то замість дефісів підкреслення і потрібен суфікс .py:

```
esptool.py --port /dev/ttyUSB0 write_flash -z 0x1000 bootloader.bin
```

Перевірити свою версію: esptool version або esptool.py version.

Перевірка після прошивки

Не «прошилося без помилок», а:

1. esptool --port /dev/ttyUSB0 verify-flash 0x10000 app.bin — звіряє вміст флешу з файлом;
2. відкрити монітор на 115200 і скинути плату кнопкою EN;
3. прочитати boot-лог (картка K6).

Прошивка вважається успішною тільки після третього пункту.

УВАГА

Дамп флешу зняти **до** прошивки, не після (картка K2). Після write-flash початкового вмісту вже немає.

K6. Читання boot-логу

Монітор на 115200 бод, натиснути EN. Перші рядки друкує ROM, і саме вони кажуть, чому плата поводить себе так, як поводить себе.

Перший рядок — головний

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
```

rst: — причина останнього скидання. Найчастіші значення для classic (повна таблиця — додаток D):

Код	Назва	Що сталося
0x1	POWERON_RESET	подано живлення або натиснуто EN. Норма
0x3	SW_RESET	скидання з коду (esp_restart)
0x5	DEEPSLEEP_RESET	прокинувся з deep sleep. Норма
0x7	TG0WDT_SYS_RESET	спрацював watchdog таймера 0
0x8	TG1WDT_SYS_RESET	спрацював watchdog таймера 1
0x9	RTCWDT_SYS_RESET	спрацював RTC watchdog
0xc	SW_CPU_RESET	скидання ядра з коду; часто після паніки
0xf	RTCWDT_BROWN_OUT_RESET	просіло живлення
0x10	RTCWDT_RTC_RESET	RTC watchdog скинув усе, разом з RTC

boot: — куди пішов чип: SPI_FAST_FLASH_BOOT — звичайний старт із флешу; DOWNLOAD_BOOT — режим прошивки: classic S3 GPIO9 притиснутий до землі, C3 GPIO9 (картка K4).

ЖИВЛЕННЯ

rst:0xf — це не програмна помилка. Це живлення. Шукати треба джерело, кабель або конденсатори, а не баг у коді. Найчастіше з’являється в момент увімкнення Wi-Fi, бо саме там піковий струм.

Далі — другий бутлоадер

```
I (29) boot: ESP-IDF v6.0.2 2nd stage bootloader
I (33) boot.esp32: SPI Flash Size : 4MB
I (52) boot: Partition Table:
I (56) boot: ## Label      Usage      Type ST Offset   Length
```

Три речі, які тут читаються безкоштовно: версія IDF, якою зібрано прошивку; обсяг флешу, який бачить бутлоадер; уся таблиця розділів із адресами. Це готова відповідь на «а що там усередині» — без розбирання дампа. Число в дужках — мілісекунди від старту. Стрибок у цьому числі показує, де саме прошивка задумалася.

Що означає тиша

Видно	Що це
нечитний набір символів	не 115200. Читається на 74880 — це ESP8266
перші рядки, далі тиша	застосунок стартував і не логує. Це може бути норма
ті самі рядки по колу	boot loop → картка K7
invalid header: 0x...	застосунку немає або адреса не та → картка K5
зовсім порожньо	немає порту чи живлення → картка K3

УВАГА

Рядки ROM завжди йдуть на 115200 — і незалежно від швидкості застосунку, і незалежно від кварцу на платі. Якщо ROM видно, а далі каша — швидкість застосунку інша, і це нормально. Якщо ж на 115200 не читається нічого, а на 74880 з’являється осмислений текст — у вас ESP8266, а не ESP32: у його ROM швидкість виходить саме такою.

K7. Guru Meditation i backtrace за 60 секунд

```
Guru Meditation Error: Core 0 panic'ed (LoadProhibited). Exception was unhandled.
Core 0 register dump:
PC      : 0x400d1234 PS      : 0x00060730 A0      : 0x800d5678
...
Backtrace: 0x400d1234:0x3ffb1f30 0x400d5678:0x3ffb1f50
```

Це не «плата зламалася». Це звіт про те, де саме програма померла.

1. Прочитати причину

Причина	Що сталося	Куди дивитися
LoadProhibited	читання за недійсною адресою	показчик NULL або звільнений
StoreProhibited	запис за недійсною адресою	те саме, але на запис
InstrFetchProhibited	перехід на недійсну адресу	зіпсований показчик на функцію
IllegalInstruction	виконання не-коду	пошкоджений стек, переповнення
LoadStoreAlignment	невирівняний доступ	читання 32 біт з непарної адреси
Interrupt wdt timeout	ISR або critical section триває задовго	код у перериванні

Task watchdog got triggered — **не паніка**: це окреме повідомлення, і система при ньому лишається живою. Воно саме називає винуватця в рядку Tasks currently running. Причина майже завжди — цикл без vTaskDelay.

Найчастіші дві — LoadProhibited і StoreProhibited, і обидві майже завжди означають одне: **розіменування показчика, який не той, що ви думаєте**. Найчастіше — результат malloc, який не перевірили.

2. Розшифрувати backtrace

Backtrace — це ланцюжок адрес. Сам по собі він нечитний; його треба перекласти в назви функцій і номери рядків. idf.py monitor робить це автоматично, якщо запущений з каталогу того самого проекту.

Вручну, коли лог знято з чужого пристрою і є .elf:

```
xtensa-esp32-elf-addr2line -pfiaC -e build/app.elf 0x400d1234 0x400d5678
```

Інструмент **свій для кожної архітектури**: **s3** — xtensa-esp32s3-elf-addr2line, **c3** та інші RISC-V — riscv32-esp-elf-addr2line.

Читати **знизу вгору**: нижні кадри — хто викликав, верхній — де впало.

3. Якщо .elf немає

Без .elf адреси перекласти нема в що: символів у прошивці немає. Лишається причина паніки і PC — цього досить, щоб відрізнити збій у власному коді від збою в стеку Wi-Fi, але не досить для рядка.

УВАГА

.elf того самого збирання, що й .bin, — єдине, що робить backtrace читним. Зберігати .elf разом із кожною прошивкою, яку віддали в поле. Перезібрати «такий самий» пізніше не вийде: адреси зсунуться.

4. Коли паніка не перша

Після паніки чип скидається — і в логу з'являється rst:0xc. Якщо причина паніки лишилася, це стає boot loop: паніка → скидання → паніка. Дивитися треба **найперший** дамп після подачі живлення, а не сотий.

Coredump у флеші (якщо ввімкнено в menuconfig) зберігає стан усіх задач, а не лише тієї, що впала: idf.py coredump-info.

K8. Топ-15 несправностей: симптом → причина → дія

Порядок — за частотою в житті, а не за складністю. Повний покажчик з посиланнями на розділи — додаток В.

#	Симптом	Найчастіша причина	Що робити
1	Порт не з'являється	кабель тільки для заряджання	інший кабель → K3
2	«Failed to connect»	плата не в download mode	BOOT + EN → K4
3	Прошилося, плата мовчить	адреса бутлоадера не та для цього чипа	звірити чип і адресу → K5
4	rst:0xf у логу	просідає живлення (brownout)	джерело, кабель, конденсатор → K13
5	Перезавантаження при вмиканні Wi-Fi	джерело не тягне піковий струм	1 A+, короткий кабель, 470 мкФ по живленню
6	Boot loop	паніка в застосунку	перший дамп після живлення → K7
7	Каша в моніторі	не та швидкість	ROM завжди 115200; 74880 = ESP8266 → K6
8	I ² C-пристрій не знаходиться	немає підтягування або земля не спільна	4.7 кОм на SDA і SCL, спільний GND
9	ADC читає дурницю	classic S2 ADC2 не працює при Wi-Fi	перенести на ADC1
10	GPIO поводить дивно при старті	це strapping-пін	інший пін → K11
11	classic GPIO 6–11 нічого не роблять	вони зайняті флешем	не чіпати ці піни взагалі
12	Працює від USB, не працює від БЖ	немає спільної землі або просадка	з'єднати GND, зміряти під навантаженням
13	Сенсор гріється, значення плывуть	подано 5 В на 3.3-вольтовий вхід	звірити datasheet, конвертер рівнів
14	Wi-Fi бачить мережу, не під'єднується	пароль, канал 12–13 або 5 ГГц	звірити SSID, канал; ESP32 не бачить 5 ГГц
15	Після erase-flash нічого немає	стерто разом із калібруванням і NVS	залити дамп → K2; надалі дамп до

Три правила, що економлять години
Спершу живлення, потім код. Більшість «плаваючих багів» — просадка напруги. Зміряти під навантаженням дешевше, ніж читати код.
Одна зміна за раз. Змінили дві речі — не знаєте, яка допомогла.
Мінімальний тест окремо. Незнайомий модуль перевіряється на голій платі з трьома дротами, а не всередині проєкту з двадцятьма.

УВАГА

Симптом «іноді працює» майже ніколи не є програмним. Дивитися в такому порядку: живлення → пайка → земля → рівні → таймінги → і лише потім код.

K9. Піни: обмеження і призначення

Повні пінаут-таблиці плат — додаток А. Тут — те, через що плати не стартують і піни не працюють.

ESP32 classic (WROOM-32, DevKitC)

Піни	Що з ними
6, 7, 8, 9, 10, 11	зайняті флешем. Не використовувати ніколи
34, 35, 36, 37, 38, 39	тільки вхід. Немає виходу і немає підтягування
0, 2, 5, 12, 15	strapping: стан при старті визначає режим завантаження
1 (TX), 3 (RX)	консоль UART0. Зайняті логом і прошивкою
12 (MTDI)	підтягнутий вгору при старті → плата не стартує взагалі
25, 26	єдині DAC-виходи
32–39	ADC1 — працює завжди
0, 2, 4, 12–15, 25–27	ADC2 — не працює при увімкненому Wi-Fi

Вільні без застережень: 4, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33.

Поширена домовленість (не апаратна прив'язка): I²C — SDA 21, SCL 22; SPI — MOSI 23, MISO 19, SCK 18, CS 5. Апаратні піни — додаток А.

ESP32-S3 (WROOM-1, DevKitC-1)

Піни	Що з ними
26–32	флеш і PSRAM. Не чіпати
33–37	додатково зайняті на модулях з Octal PSRAM (N16R8)
0, 3, 45, 46	strapping
19, 20	native USB (D–, D+)
1–10	ADC1
11–20	ADC2

УВАГА

S3

Комбінація GPIO46 = 1 разом із GPIO0 = 0 недійсна і призводить до непередбачуваної поведінки. Якщо на цих пінах щось висить — знімати.

ESP32-C3 (MINI-1, SuperMini)

Піни	Що з ними
12–17	флеш. Не чіпати
2, 8, 9	strapping
18, 19	USB-Serial-JTAG. Перевизначили — втратили налагодження
0–4	ADC1
5	ADC2

Вхід у бутлоадер: GPIO9 притиснутий до землі при скиданні, GPIO8 при цьому має бути високим. Комбінація GPIO8 = 0 і GPIO9 = 0 недійсна.

Правило, що економить найбільше часу
Пінаут плати важливіший за пінаут чипа. Виробник плати міг вивести не всі піни, підписати їх по-своєму, повісити світлодіод або підтягувальний резистор. Те, що пін є на схемі чипа, не означає, що він вільний на цій платі.

НЕЗВОРОТНЕ

Пін із позначкою «тільки вхід» не стає виходом від налаштування в коді. Спроба керувати з нього навантаженням — це або нічого, або пошкоджений пін.

K10. Шпаргалка команд

Синтаксис esptool v5 (дефіси, без .py). Для v4 — підкреслення і суфікс .py: esptool.py write_flash. Перевірити своє: esptool version.

esptool

```
esptool --port /dev/ttyUSB0 chip-id          # що за чип і ревізія
esptool --port /dev/ttyUSB0 flash-id         # обсяг і виробник флешу
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump.bin # повний дамп
esptool --port /dev/ttyUSB0 write-flash -z 0x10000 app.bin # залити
esptool --port /dev/ttyUSB0 verify-flash 0x10000 app.bin # звірити
esptool --port /dev/ttyUSB0 erase-flash      # стерти все (Δ див. K2)
esptool --port /dev/ttyUSB0 --baud 115200 ... # повільніше, надійніше
esptool merge-bin -o all.bin --flash-mode dio \
  0x1000 boot.bin 0x8000 pt.bin 0x10000 app.bin # 0x1000 → classic/S2; інші чипи — див. таблицю
```

idf.py

```
idf.py create-project my-project # новий проєкт (назва латиницею)
idf.py set-target esp32s3        # Δ стирає sdkconfig
idf.py menuconfig               # налаштування
idf.py build                     # зібрати
idf.py -p /dev/ttyUSB0 flash     # залити
idf.py -p /dev/ttyUSB0 monitor   # монітор з розшифровкою backtrace
idf.py -p /dev/ttyUSB0 flash monitor # найчастіша команда
idf.py fullclean                 # коли збирання поводитьсся незрозуміло
idf.py size                     # скільки зайнято флешу і RAM
idf.py coredump-info             # розбір coredump із флешу
```

Монітор

idf.py monitor: **вийти** — Ctrl+J. **Скинути плату** — Ctrl+T, потім Ctrl+R.

```
minicom -D /dev/ttyUSB0 -b 115200 # вийти: Ctrl+A, потім X
screen /dev/ttyUSB0 115200         # вийти: Ctrl+A, потім K
picocom -b 115200 /dev/ttyUSB0    # вийти: Ctrl+A, потім Ctrl+X
```

Порт

```
ls /dev/ttyUSB* /dev/ttyACM* # Linux: що є
dmesg | tail                 # що ядро побачило при під'єднанні
sudo usermod -aG dialout $USER # права; далі ПЕРЕЗАЙТИ в систему
lsof /dev/ttyUSB0            # хто тримає порт
```

/dev/ttyUSB* — зовнішній міст (CP2102, CH340). /dev/ttyACM* — native USB S3 C3.

Адреси

Що	classic, S2	S3, C3, C6, H2	P4, C5, H4
bootloader	0x1000	0x0	0x2000
partition table	0x8000	0x8000	0x8000
застосунок	0x10000	0x10000	0x10000
зібраний merge-bin	0x0	0x0	0x0

K11. Ніколи

Кожен пункт нижче — незворотний або майже незворотний. Прочитати до, а не після. Решта помилок лікується.

НЕЗВОРОТНЕ

Не палити eFuse наосліп. esepfuse записує біти, які **фізично не скидаються назад**. Помилковий біт може назавжди відібрати JTAG, download mode або можливість перепрошивки. Не запускати команду esepfuse burn-*, поки не зрозуміло дослівно, що робить кожен її аргумент.

НЕЗВОРОТНЕ

Не вмикати Flash Encryption і Secure Boot «щоб подивитися». В release-режимі це односторонні двері: чип перестає приймати непідписані прошивки, а флеш стає нечитним поза цим конкретним чипом. Дамп, знятий до цього, ще можна залити; знятий після — ні. Пробувати тільки на платі, яку не шкода.

НЕЗВОРОТНЕ

Не стирати флеш без дампа. erase-flash знищує NVS разом із калібруванням радіо, збереженими креденцями і конфігурацією конкретного пристрою. Це не завжди відновлюється перезбиранням прошивки. Спершу картка K2.

НЕЗВОРОТНЕ

Не подавати 5 В на GPIO. Логіка ESP32 — 3.3 В. П'ять вольтів вбивають пін, іноді весь порт, іноді чип. Найчастіші джерела: HC-SR04 (вихід ЕСН0), релеїні модулі з VCC 5 В, «сумісні з Arduino» датчики. Дільник або конвертер рівнів — обов'язково. Виняток єдиний: пін 5V/VIN — це вхід стабілізатора, туди 5 В можна.

НЕЗВОРОТНЕ

classic Не чіпати GPIO 6, 7, 8, 9, 10, 11. Вони з'єднані з мікросхемою флешу на самому модулі. Будь-яка спроба їх використати підвищує чип або псує вміст флешу. На пінауті вони часто виведені — це не означає, що вони вільні.

Не незворотне, але дороге

Не паяти під живленням. На жалі незаземленого паяльника може бути наведений потенціал відносно землі плати, і цього досить, щоб пробити вхід. Живлення від'єднується завжди.

Не під'єднувати земляний щуп осцилографа абиде. У приладах із мережевим живленням земля щупа — це земля розетки. Одне невдале дотикання коротить живлення схеми через прилад.

Не тримати strapping-піни навантаженими під час старту. **classic** GPIO0, GPIO2, GPIO5, GPIO12, GPIO15. Підтягнутий не в той бік GPIO12 перемикає напругу флешу і не дає платі стартувати взагалі.

Не міняти дві речі одночасно. Не незворотнo, але з'їдає більше часу, ніж усе перелічене вище разом.

K12. Польовий комплект і закупівля

Актуальні ціни, наявність і конкретні магазини — у датованому вкладиші ринку, не тут. Тут — що саме шукати і чому.
Мінімум, без якого не працює

Позиція	На що дивитися
Плата ESP32 DevKit	38 пінів, USB-UART CP2102 або CH9102
Кабель USB data	перевірити на телефоні перед виїздом
Мультиметр	прозвонка зі звуком, вимірювання постійної напруги
Паяльник із терморегулятором	60 Вт, жало «скіс» 2–3 мм
Припій і флюс	0.5–0.8 мм зі свинцем; флюс-гель у шприці
Макетка + Dupont	усі три види: тато-тато, тато-мама, мама-мама
USB-хаб із власним живленням	знімає половину «незрозумілих глюків»

Те, що окупається на другому пристрої

Позиція	Навіщо
Логічний аналізатор 8 каналів	бачити I ² C і SPI наживо замість гадання
Лабораторний БЖ з обмеженням струму	обмеження струму рятує плату при КЗ
USB-тестер напруги і струму	ловить просадки і рахує споживання
Термофен	зняти модуль, не відірвавши площадки
Пінцет, бокорізи, оплітка	зняти зайвий припій, а не розмазати
Набір резисторів і конденсаторів	4.7 кОм і 10 кОм, 100 нФ і 470 мкФ — найчастіші

Витратне, яке завжди закінчується не вчасно

Гребінки (male і female), термоусадка кількох діаметрів, дроти 22–26 AWG трьох кольорів, ізоляційна стрічка, спирт і безворсові серветки, запасні кабелі USB.

закупівля

Три позиції, на яких не варто економити. Мультиметр — дешевий бреше на межах вимірювання. Паяльник — без терморегулятора або не гріє, або палить площадки. Кабель — половина «плата не визначається» саме тут.

Три позиції, де дешеве нормальне. Макетка, Dupont-дроти, гребінки. Брати з запасом: вони губляться і псуються.

Що покласти в сумку, яка їде в поле

Плата з уже залитою робочою прошивкою · перевірений кабель · павербанк · мультиметр · викрутки · ніж · ізоляційна стрічка · термоусадка · запасні гребінки · роздруковані й заламіновані картки K1–K15.
Ноутбук із офлайн-комплектом (додаток F): встановлений тулчейн, завантажена документація, закешовані бібліотеки. У полі інтернету немає, і саме там він потрібен найбільше.

K13. Живлення: пошук несправності

Більшість «плаваючих багів» — це живлення. Симптоми виглядають програмними, тому люди читають код. Ця картка — перше, що робиться замість цього.

Ознаки, що це живлення

- `rst:0xf` у boot-лозі — **гарантовано живлення**, не код
- перезавантаження саме при вмиканні Wi-Fi, двигуна, реле
- «іноді працює», залежить від кабелю чи розетки
- працює від USB, не працює від зовнішнього джерела
- `MD5 does not match` при прошивці
- датчики віддають шум

Чотири вимірювання

Мультиметр, у цьому порядку.

#	Що міряти	Норма	Якщо ні
1	3V3–GND, прозвонка, живлення вимкнене	не дзвенить	КЗ — не вмикати, шукати
2	Вхідна напруга на роз’ємі	5 В ±5 % (від USB)	джерело, кабель, роз’єм
3	3V3 на холостому ході	3.2–3.4 В	стабілізатор
4	3V3 під навантаженням, Wi-Fi увімкнений	не нижче 3.0 В	 причина знайдена

Четвертий рядок — головний. Три перші часто в нормі, а провал видно лише під навантаженням.

Порядок лікування, від дешевого

1. **Інший кабель.** Короткий, товстий, завідомо data. Розв’язує половину випадків.
2. **Порт напряму,** без хаба. Або хаб із **власним живленням.**
3. **Конденсатор 470 мкФ** між 3V3 і GND поруч із модулем. Копійки, ставиться за хвилину.
4. **Керамічний 100 нФ** біля кожної мікросхеми.
5. **Окреме джерело від 1 А.** Не 500 мА: платити треба за піки.
6. **Живлення 3.3 В напряму,** мінаючи бортовий стабілізатор — коли він слабкий на клоні.
7. **Знизити пікове споживання:** потужність передавача, частота ядра.

Скільки треба струму

Джерело для ESP32 з Wi-Fi — **щонайменше 1 А**, навіть якщо середнє споживання сто міліампер. Передавач вмикається імпульсами, і платить джерело саме за них.

Двигун, серво, реле, підсвітка TFT — **окреме живлення**, спільна земля.

НЕЗВОРОТНЕ

Не вмикати детектор brownout у menuconfig, «щоб не заважав». Перезавантаження зникнуть, захист теж — і замість чесного скидання прийдуть пошкоджені NVS і збої, які неможливо пояснити.

Це діагностика, а не перешкода.

Автономне живлення

- **Дільник вимірювання напруги вмикати транзистором** — інакше він стає головним споживачем уві сні
- **Buck-boost, не LDO** — LDO відсікає половину ємності 18650
- **Плата розробки для сну не годиться:** USB-міст, стабілізатор і світлодіод не сплять. 20 мА замість 20 мкА — це вони
- **Не заряджати літій нижче 0 °C**

Швидка таблиця

Симптом	Найімовірніше
<code>rst:0xf</code>	кабель, хаб, немає конденсатора
Перезавантаження при Wi-Fi	джерело не тягне піків
Перезавантаження при двигуні	немає окремого живлення
Стабілізатор гарячий	перевантаження або пробій
Працює від USB, не від БЖ	немає спільної землі
Сон 20 мА замість 20 мкА	плата розробки, а не чип

K14. Сумісність логічних рівнів

Логіка ESP32 — 3.3 В. Половина модулів «для Arduino» — 5 В. Зовні роз’єми однакові.

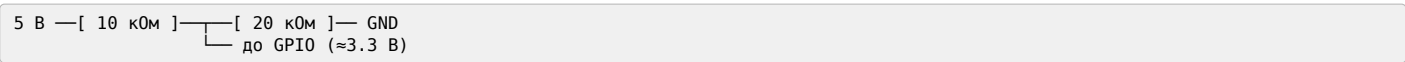
НЕЗВОРОТНЕ
5 В на GPIO вбивають пін, іноді чип. Єдиний виняток — пін 5V/VIN: це вхід стабілізатора, а не вхід у чип.

Живлення і рівні — різні питання
Модуль може житися від 5 В і мати виводи на 3.3 В. Або житися від 3.3 В і не сприймати 3.3 В як одиницю.
Перевіряти рівні на сигнальних виводах у datasheet, а не вгадувати за написом живлення.

Що на платі модуля	Живлення	Сигнали
Немає стабілізатора	3.3 В	3.3 В — прямо
Є стабілізатор 5→3.3	5 В	перевіряти
Є стабілізатор і конвертер	5 В	5 В — потрібен конвертер

Два напрямки
ESP32 → пристрій на 5 В. Часто працює прямо: більшість 5-вольтових входів бере 3.3 В за одиницю. Не завжди — звірятися.
Пристрій на 5 В → ESP32. Зниження **обов’язкове**.

Чим знижувати
Дільник напруги — два резистори. Дешево.



Годиться: повільні однонаправлені сигнали. **Не годиться:** I²C (двонаправлений), швидкий SPI (завалює фронти).
Модуль конвертера рівнів на польових транзисторах — двонаправлений, працює з I²C, коштує копійки. LV до 3.3 В, HV до 5 В, землі з’єднані.
Тримати пару в шухляді: потреба виникає раптово.

Часті винуватці 5 В

Модуль	Де 5 В
HC-SR04	вивід ЕСНО
Релейні модулі	вхід керування і живлення котушки
Дисплеї «для Arduino»	сигнальні лінії
MAX485, TJA1050, MCP2551	вивід RX трансивера
Логіка 74НС при живленні 5 В	усі виходи

Спільна земля

НЕЗВОРОТНЕ
Усі з’єднані пристрої мусять мати **спільну землю** — крім тих, де стоїть гальванічна розв’язка (оптопара, ізольований трансивер).
Симптом відсутньої землі: живлення є, обміну немає, або обмін із випадковими помилками.
Зворотна помилка: з’єднати землі там, де розв’язка стоїть саме для того, щоб їх не з’єднувати. Тоді ізоляція перестає існувати, а схема виглядає робочою.

Перевірка за десять секунд
Мультиметром, до з’єднання:
1. Напруга на виводі живлення модуля — та, що очікуєте?
2. Напруга на сигнальному виводі в спокої — 3.3 чи 5?
3. Прозвонка землі між модулем і платою.
Три вимірювання дешевші за спалений пін.

K15. Чек-лист серійної прошивки

Прошити партію так, щоб через півроку можна було відповісти, що в кожному пристрої.

Підготовка (один раз на партію)

- ☐ Зібрано **один образ** через merge-bin — одна команда, одна адреса, нема чого переплутати
- ☐ --flash-mode, --flash-size, --flash-freq збігаються з sdkconfig проекту
- ☐ Образ перевірено на одній платі повністю: прошивка → boot-лог → функціональний тест
- ☐ Збережено: .bin, .elf **того самого збирання**, sdkconfig, версія ESP-IDF, версії компонентів
- ☐ Заведено журнал партії
- ☐ Живлення для прошивки — окреме, з запасом, не від ноутбука

```
esptool merge-bin -o vyrib-v1.4.bin --flash-mode dio \
0x1000 bootloader.bin 0x8000 partition-table.bin 0x10000 app.bin
```

На кожну плату

☐	Крок	Чим
☐	1. Прошити	write-flash -z 0x0 vyrib-v1.4.bin
☐	2. Звірити	verify-flash 0x0 vyrib-v1.4.bin
☐	3. Записати унікальне	NVS: номер, ключі, калібрування
☐	4. Зняти MAC	єдиний надійний ідентифікатор плати
☐	5. Прочитати boot-лог	пристрій справді стартує
☐	6. Функціональна перевірка	те, заради чого виріб існує
☐	7. Маркувати	номер, версія, дата — на видний бік
☐	8. Записати рядок у журнал	

Кроки 2, 5 і 6 — обов’язкові, не опції. «Прошилося без помилок» ловить не все: крок 6 виявляє справний образ на платі з непропаяним модулем.

Журнал партії

№	MAC	Версія	Дата	Контроль	Примітка
0041	A0:B7:…:14	v1.4	2026-08-26	ОК	
0043	A0:B7:…:31	v1.4	2026-08-26	брак	не стартує

Спільне і унікальне

НЕЗВОРОТНЕ

Спільний образ ≠ спільна конфігурація.

Залити двадцятьом платам однаковий NVS означає двадцять пристроїв з однаковою особистістю: однаковий серійний номер, однакові ключі. Виявиться це пізно, у полі, і виглядатиме як містика — два пристрої «крадуть» з’єднання одне в одного.

Один ключ на всі також означає, що захоплення одного компрометує всі.

Якщо треба лише відрізнати пристрої — **беріть MAC**: він унікальний від заводу і не потребує нічого.

```
nvs_partition_gen.py generate config-0042.csv nvs-0042.bin 0x6000
esptool --port PORT write-flash 0x9000 nvs-0042.bin
```

Якщо плата не прошивається

Не зупиняти партію. Відкласти, позначити, продовжити з рештою.

Найчастіші причини саме в партії: непропаяний модуль, замикання при монтажі, плата іншої ревізії в тій самій коробці.

Прискорення і журнал партії

Кілька портів паралельно, оснастка з рого pins, ведення журналу і відновлення після браку — розділ [21]. Тут лише те, що робиться руками над кожною платою.

Дві дрібниці, що економлять найбільше: скрипт із set -e (інакше збій губиться в потоці виводу) і звертання до порту через /dev/serial/by-id/, а не ttyUSB0 — номери плутаються після кожного перевикання.

Незворотне

НЕЗВОРОТНЕ

- erase-flash — тільки після дампа (картка K2)
- espefuse burn-* — не запускати, доки не зрозуміло дослівно, що робить кожен аргумент
- Flash Encryption і Secure Boot у release — незворотні. На партію — лише після випробування на платі, яку не шкода